EJERCICIO 8

Árbol abarcador de costo mínimo con Prim

Grafo: ver ej8_grafo.png. Árbol resultante: ver ej8_arbol.png.

En los órdenes, V es la cantidad de vértices y E la cantidad de aristas.

ESTRUCTURAS, CONCEPTOS Y ALGORITMOS

- Estructura: grafo no dirigido y ponderado. Cada zona es un vértice y cada camino posible una arista con los km como peso. Es no dirigido porque un camino de tierra se recorre en los dos sentidos y cuesta lo mismo. Hay distancia entre todo par de zonas, así que es completo: 6 vértices y 15 aristas.
- Concepto: lo que se busca es un árbol abarcador de costo mínimo. Abarcador porque tiene que llegar a todas las zonas. Árbol porque con n zonas alcanzan n - 1 caminos, y cualquier camino de más cerraría un ciclo. De costo mínimo porque la suma de los km tiene que ser la menor posible.
- Algoritmos: Prim y Kruskal. Los dos son golosos y dan un árbol de costo mínimo. Prim hace crecer un solo árbol desde un vértice y en cada paso agrega la arista más barata que une el árbol con un vértice que todavía no está. Kruskal mira las aristas de menor a mayor costo y agrega cada una si une dos componentes distintas. Este ejercicio pide Prim.

REPRESENTACIÓN GRÁFICA

Ver ej8_grafo.png.

Lista de adyacencias:

```
    Zona 1 -> Zona 2(2), Zona 3(5), Zona 4(1), Zona 5(7), Zona 6(2)
    Zona 2 -> Zona 1(2), Zona 3(1), Zona 4(3), Zona 5(5), Zona 6(6)
    Zona 3 -> Zona 1(5), Zona 2(1), Zona 4(2), Zona 5(3), Zona 6(5)
    Zona 4 -> Zona 1(1), Zona 2(3), Zona 3(2), Zona 5(7), Zona 6(1)
    Zona 5 -> Zona 1(7), Zona 2(5), Zona 3(3), Zona 4(7), Zona 6(4)
    Zona 6 -> Zona 1(2), Zona 2(6), Zona 3(5), Zona 4(1), Zona 5(4)
```

Cada arista aparece dos veces, una en la lista de cada extremo.

PRIM PASO A PASO

U es el conjunto de zonas que ya están en el árbol. Se empieza desde Zona 1. En cada paso se miran las aristas que van de U al resto y se elige la de menor costo. Si hay empate, se queda la primera que se encontró (recorriendo U en el orden en que se agregaron y las adyacencias en el orden de la lista).

```
Paso 1: U = {1}
        candidatas: (1,2) 2, (1,3) 5, (1,4) 1, (1,5) 7, (1,6) 2
        se elige (1,4) costo 1                          -> U = {1, 4}
Paso 2: U = {1, 4}
        desde 1: (1,2) 2, (1,3) 5, (1,5) 7, (1,6) 2
        desde 4: (4,2) 3, (4,3) 2, (4,5) 7, (4,6) 1
        se elige (4,6) costo 1                          -> U = {1, 4, 6}
Paso 3: U = {1, 4, 6}
        desde 1: (1,2) 2, (1,3) 5, (1,5) 7
        desde 4: (4,2) 3, (4,3) 2, (4,5) 7
        desde 6: (6,2) 6, (6,3) 5, (6,5) 4
        empatan (1,2) y (4,3) con costo 2: se elige (1,2), la primera que se encontró
                                                        -> U = {1, 4, 6, 2}
Paso 4: U = {1, 4, 6, 2}
        desde 1: (1,3) 5, (1,5) 7
        desde 4: (4,3) 2, (4,5) 7
        desde 6: (6,3) 5, (6,5) 4
        desde 2: (2,3) 1, (2,5) 5
        se elige (2,3) costo 1                          -> U = {1, 4, 6, 2, 3}
Paso 5: U = {1, 4, 6, 2, 3}
        hacia 5: (1,5) 7, (4,5) 7, (6,5) 4, (2,5) 5, (3,5) 3
        se elige (3,5) costo 3                          -> U = {1, 4, 6, 2, 3, 5}
```

U ya tiene las 6 zonas: termina.

RESULTADO

Ver ej8_arbol.png.

Camino	km
Zona 1 - Zona 4	1
Zona 4 - Zona 6	1
Zona 1 - Zona 2	2
Zona 2 - Zona 3	1
Zona 3 - Zona 5	3
Total	8

La mejor opción es construir esos 5 caminos, con 8 km en total.

Si en el paso 3 se elige (4,3) en lugar de (1,2), el paso siguiente agrega (3,2), de costo 1, y el total también es 8. El árbol puede cambiar con los empates, pero el costo mínimo es siempre el mismo.

PARTE 2 - PRIM EN PSEUDOCÓDIGO Y CASOS DE PRUEBA

prim(grafo, origen)

Lenguaje natural:
Se arma un grafo nuevo con todos los vértices y sin aristas, que va a ser el árbol. U empieza solo con el origen y "resto" con todos los demás. Mientras quede algún vértice en el resto, se busca la arista de menor costo con un extremo en U y el otro en el resto (buscarMin), se la agrega al árbol y su extremo pasa del resto a U. Si buscarMin no encuentra ninguna arista, lo que queda en el resto no se puede alcanzar (el grafo no es conexo) y se corta.

buscarMin recorre las aristas de cada vértice de U y se queda con la más barata cuyo otro extremo está en el resto. Así no pregunta por pares de vértices que no tienen arista.

Estructuras: el grafo no dirigido, dos conjuntos de vértices que conservan el orden en que se agregan (agregar, quitar y contiene en O(1)) y el grafo del resultado.

Precondiciones:
- El grafo es no dirigido y los pesos son números.
- El origen existe en el grafo. Si no existe, se lanza un error.

Postcondiciones:
- El grafo original no cambia.
- Devuelve un grafo nuevo con todos los vértices del original.
- Si el grafo es conexo, las aristas del resultado forman un árbol abarcador de costo mínimo: n - 1 aristas, sin ciclos, y la suma de sus pesos es la menor posible.
- Si no es conexo, devuelve el árbol de costo mínimo de la componente del origen y los demás vértices quedan sin aristas.

Pseudocódigo:

```
prim(grafo, origen) devuelve GrafoNoDirigido
Comienzo
    SI NO existeVertice(grafo, origen) ENTONCES
        error "el vértice origen no existe en el grafo"
    FIN SI

    arbol <- grafo no dirigido vacío
    PARA CADA v EN vertices(grafo) HACER
        arbol.agregarVertice(v)
    FIN PARA

    U <- conjunto vacío que conserva el orden de ingreso
    resto <- conjunto con vertices(grafo), en el mismo orden
    U.agregar(origen)
    resto.quitar(origen)

    MIENTRAS resto NO ESTA VACIO HACER
        arista <- buscarMin(grafo, U, resto)
```

```
        SI arista ES NULO ENTONCES
            salir del MIENTRAS              // lo que queda en resto no es alcanzable
        FIN SI
        arbol.agregarArista(arista.origen, arista.destino, arista.peso)
        U.agregar(arista.destino)
        resto.quitar(arista.destino)
    FIN MIENTRAS
    devolver arbol
Fin

buscarMin(grafo, U, resto) devuelve Arista
Comienzo
    mejor <- nulo
    min <- infinito
    PARA CADA u EN U HACER
        PARA CADA arista EN adyacencias(grafo, u) HACER
            // en la lista de u, arista.origen es u
            SI arista.destino ESTA EN resto Y arista.peso < min ENTONCES
                min <- arista.peso
                mejor <- arista
            FIN SI
        FIN PARA
    FIN PARA
    devolver mejor                          // nulo si ninguna arista cruza
Fin
```

Orden del tiempo de ejecución:
- buscarMin mira cada arista de los vértices de U una vez: O(E). Además, en esta implementación adyacencias(grafo, u) primero busca el vértice, que cuesta O(V), así que buscarMin es $O(V^2 + E)$.
- El MIENTRAS da V - 1 vueltas: $O(V \cdot (V^2 + E))$. En un grafo completo como este ($E \sim V^2$) es $O(V^3)$.
- Con dos vectores (para cada vértice fuera del árbol, el vértice de U más cercano y su costo) queda $O(V^2)$, y con una cola de prioridad $O(E \log V)$.

PRUEBA PASO A PASO CON 4 ZONAS

Escenario reducido: solo las zonas 1 a 4.

```
    Zona 1 -> Zona 2(2), Zona 3(5), Zona 4(1)
    Zona 2 -> Zona 1(2), Zona 3(1), Zona 4(3)
    Zona 3 -> Zona 1(5), Zona 2(1), Zona 4(2)
    Zona 4 -> Zona 1(1), Zona 2(3), Zona 3(2)

prim(grafo, "Zona 1"):
    arbol <- {1, 2, 3, 4} sin aristas
    U <- {1}      resto <- {2, 3, 4}

Vuelta 1 del MIENTRAS: buscarMin(grafo, {1}, {2, 3, 4})
    u = 1: (1,2) 2: 2 está en resto y 2 < inf       -> min = 2, mejor = (1,2)
           (1,3) 5: 5 < 2 es falso                   -> sin cambios
           (1,4) 1: 1 < 2                            -> min = 1, mejor = (1,4)
    devuelve (1,4)
    arbol <- arbol + (1,4)     U <- {1, 4}     resto <- {2, 3}

Vuelta 2 del MIENTRAS: buscarMin(grafo, {1, 4}, {2, 3})
    u = 1: (1,2) 2: 2 < inf                          -> min = 2, mejor = (1,2)
           (1,3) 5: 5 < 2 es falso                   -> sin cambios
           (1,4): 4 no está en resto                 -> se saltea
    u = 4: (4,1): 1 no está en resto                 -> se saltea
           (4,2) 3: 3 < 2 es falso                   -> sin cambios
           (4,3) 2: 2 < 2 es falso (empate)          -> sin cambios
    devuelve (1,2)
    arbol <- arbol + (1,2)     U <- {1, 4, 2}  resto <- {3}

Vuelta 3 del MIENTRAS: buscarMin(grafo, {1, 4, 2}, {3})
    u = 1: (1,3) 5: 5 < inf                          -> min = 5, mejor = (1,3)
    u = 4: (4,3) 2: 2 < 5                            -> min = 2, mejor = (4,3)
    u = 2: (2,3) 1: 1 < 2                            -> min = 1, mejor = (2,3)
    (las demás aristas van a vértices que ya están en U)
    devuelve (2,3)
```

```
    arbol <- arbol + (2,3)     U <- {1, 4, 2, 3}  resto <- {}
```

resto está vacío: termina y devuelve el árbol.

Resultado: (1,4) 1 + (1,2) 2 + (2,3) 1 = 4 km, con 3 aristas para 4 zonas. Es correcto: las otras tres aristas cuestan 5, 3 y 2, y cambiar cualquiera de las elegidas por una de ellas da un total mayor o igual.

CASOS DE PRUEBA

costo(arbol) es la suma de los pesos de sus aristas.

- Grafo del ejercicio (6 zonas) desde Zona 1: el árbol tiene las 6 zonas, 5 aristas, es conexo, no tiene ciclos y cuesta 8 km.

```
    casoGrafoDelEjercicio
    Comienzo
        g <- grafo de las 6 zonas
        arbol <- prim(g, "Zona 1")
        verificar cantidadDeVertices(arbol) = 6
        verificar cantidadDeAristas(arbol) = 5
        verificar esConexo(arbol) Y NO tieneCiclos(arbol)
        verificar costo(arbol) = 8
    Fin
```

- El costo no depende del origen: desde cualquier zona da 8 km.

```
    casoCualquierOrigen
    Comienzo
        g <- grafo de las 6 zonas
        PARA CADA v EN vertices(g) HACER
            verificar costo(prim(g, v)) = 8
        FIN PARA
    Fin
```

- Escenario reducido de 4 zonas: aristas (1,4), (1,2) y (2,3), costo 4.

```
    casoCuatroZonas
    Comienzo
        g <- grafo de las zonas 1 a 4
        arbol <- prim(g, "Zona 1")
        verificar existeArista(arbol, 1, 4) Y existeArista(arbol, 1, 2) Y existeArista(arbol, 2, 3)
        verificar cantidadDeAristas(arbol) = 3
        verificar costo(arbol) = 4
    Fin
```

- No modifica el grafo original: sigue con 6 vértices y 15 aristas.

```
    casoNoModificaElOriginal
    Comienzo
        g <- grafo de las 6 zonas
        prim(g, "Zona 1")
        verificar cantidadDeVertices(g) = 6 Y cantidadDeAristas(g) = 15
    Fin
```

- Origen inexistente: se lanza un error.

```
    casoOrigenInexistente
    Comienzo
        g <- grafo de las 6 zonas
        verificar que prim(g, "Zona 9") lanza un error
    Fin
```

- Un solo vértice: el árbol tiene ese vértice y ninguna arista.
- Grafo no conexo (por ejemplo A-B y C-D sueltos): desde A el resultado tiene los 4 vértices pero solo la arista A-B. C y D quedan sin aristas.
- Grafo que ya es un árbol: devuelve las mismas aristas.
- Todas las aristas con el mismo peso w: n - 1 aristas y costo (n - 1) · w.
- buscarMin con U = {1} y resto = {2, ..., 6} devuelve (1,4), la menor que cruza. Si ninguna arista une U con el resto, devuelve nulo.

- Empate: con U = {1, 4, 6}, buscarMin devuelve (1,2) y no (4,3), porque es la primera de costo 2 que encuentra.
- Prim sin origen sobre un grafo vacío devuelve nulo, porque no hay desde dónde empezar.